

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
ESTUDIOS GENERALES CIENCIAS

TÉCNICAS DE PROGRAMACIÓN
Semestre académico 2025-2

GUÍA DE CREACIÓN, EJECUCIÓN Y DEPURACIÓN DE UN PROGRAMA EN C/C++

PROYECTO EN CLion

En el Entorno Integrado de Desarrollo CLion de la compañía JetBrains, un proyecto es un conjunto de archivos que conforman una aplicación (programa). No es una buena práctica de programación pensar que un programa se deba desarrollar en un solo archivo. La razón es muy simple, en la vida de un desarrollador de software se presentarán infinidad de oportunidades de solucionar problemas de información por medio de programas. Es muy probable que estos problemas sean muy distintos entre sí, sin embargo también es muy probable que para solucionarlos empleemos herramientas comunes a estos problemas. Por ejemplo, calcular el promedio de un conjunto de datos, ordenar los registros de un conjunto de personas o productos, dado un código buscar la información de un producto, etc. son tareas que se repiten comúnmente en la mayoría de los programas.

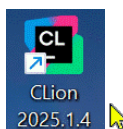
Como desarrolladores de software no podemos darnos el lujo de tener que volver a escribir el código para ordenar datos o calcular un promedio cada vez que tengamos que crear un programa para solucionar un problema. Si la primera vez que necesitemos escribir el código para calcular un promedio lo hacemos en un archivo independiente al programa principal y lo incorporamos al "proyecto" que solucionará el problema específico, la siguiente vez que estemos solucionando otro problema y necesitemos calcular un promedio, no necesitaremos volverlo a escribir sino que buscaremos el archivo que lo contenga y simplemente lo incorporaremos al nuevo proyecto. Esto no se podría hacer si todo el programa estuviese contenido en un solo archivo.

La buena práctica de reutilizar código nos ahorrará mucho tiempo en el desarrollo de una aplicación.

En esta guía nos dedicaremos a ver cómo podemos crear un proyecto en CLion, compilarlo, ejecutarlo y depurarlo. La reutilización de código no será desarrollada aún en esta guía ya que se requieren algunos conceptos previos que serán tratados más adelante.

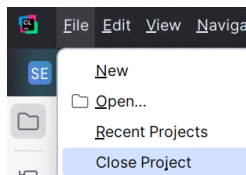
CREACIÓN DE UN PROYECTO

Empecemos por ejecutar el IDE de CLion. Para esto busque el ícono correspondiente en el escritorio de su computador y dele dos golpes con el mouse:

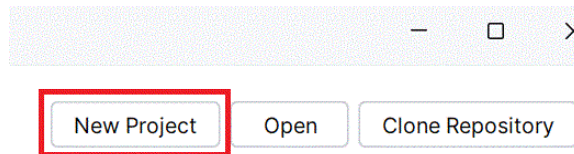


Luego que el programa se haya cargado empezaremos creando un nuevo proyecto, para esto debemos seguir los siguientes pasos:

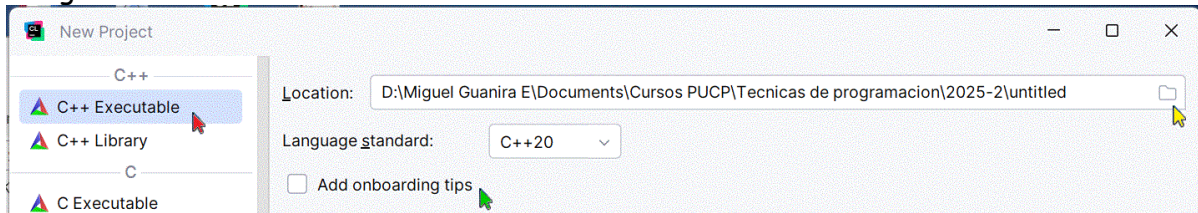
Si al ejecutar CLion observa que hay un proyecto cargado ciérrelo, para esto vaya a la barra de herramientas, elija la opción File y de allí a la opción "Close Project" como se muestra a continuación.



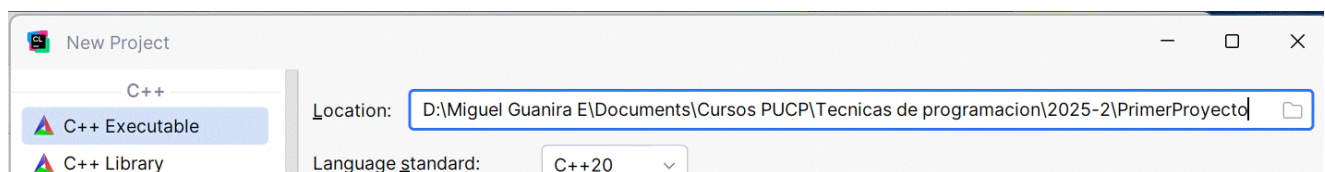
Si no había un proyecto abierto o si lo cerró, vaya al botón "New Project", como se aprecia en la figura siguiente y presiónelo.



Inmediatamente aparecerá una ventana en la que debemos indicar el lenguaje de programación (C++ Executable) indicado con la flecha  y luego defina la carpeta donde se colocará el proyecto y el nombre del proyecto (Location) indicada con la flecha , como se aprecia en la figura siguiente:

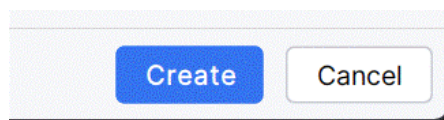


Por ejemplo.

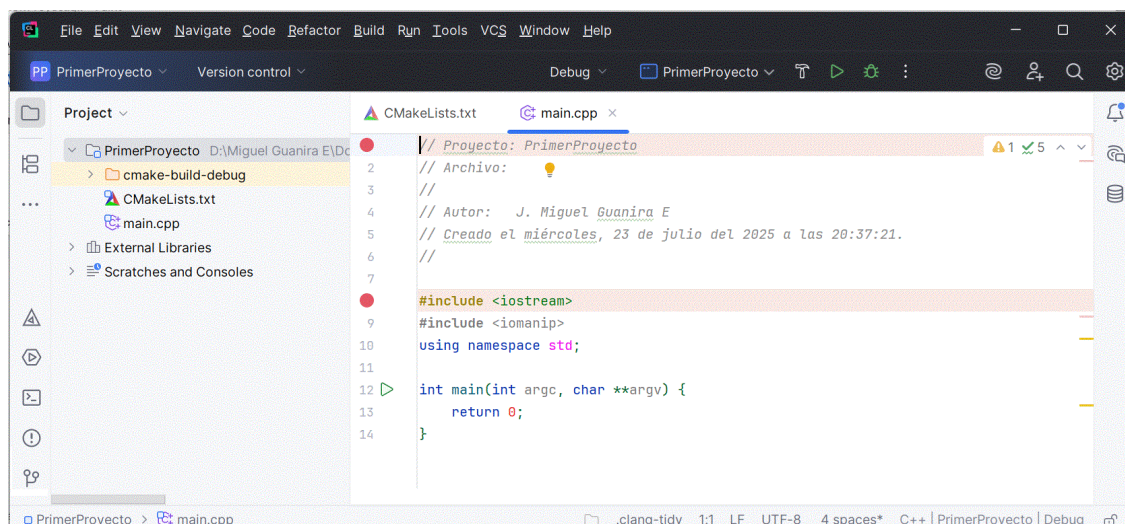


Antes de proseguir fíjese en la opción "Add onboarding tips", indicada con la flecha , si la opción está seleccionada puede desmarcarla, sino lo hace entonces el proyecto se creará en modo "ayuda" por decirlo así, destinado a principiantes que no se familiarizan con el lenguaje C ni C++.

En seguida presione el botón Create en la parte inferior derecha de la ventana.



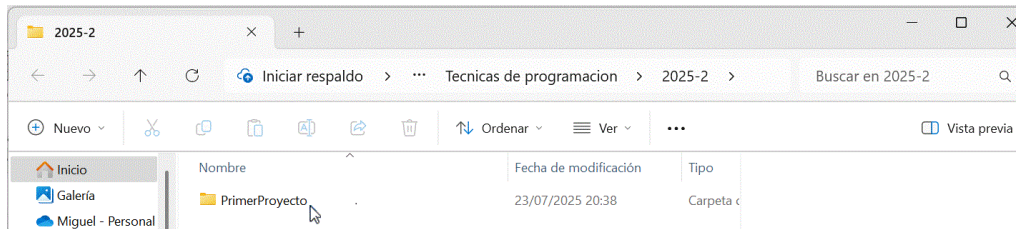
La ventana se cerrará y usted podrá observar que el proyecto se ha creado, la figura siguiente le indicará que esto se ha hecho correctamente, la imagen muestra un pantalla similar de lo que verá al realizar la tarea, esta vista puede configurarse.



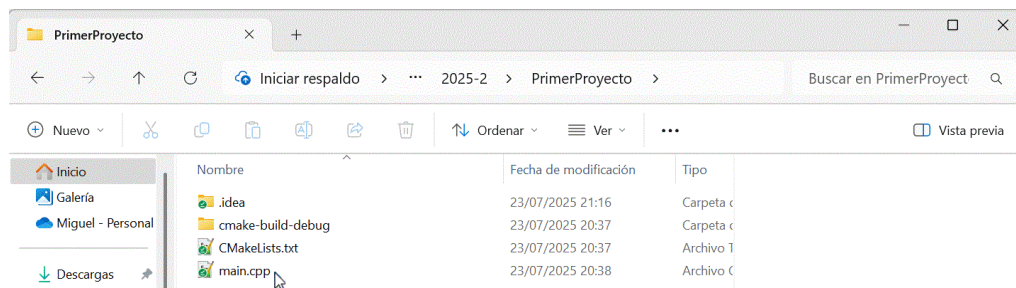
Carpeta del proyecto

Al llegar a este punto se habrá creado una carpeta con los archivos necesarios para empezar a escribir y ejecutar nuestro programa. Debemos ubicar esta carpeta porque en muchos casos deberemos realizar tareas en ella que no se pueden realizar desde el IDE del CLion, por esto nos detendremos un momento aquí.

Abramos el explorador de archivos y entremos a la carpeta que ha creado el CLion como se muestra a continuación:



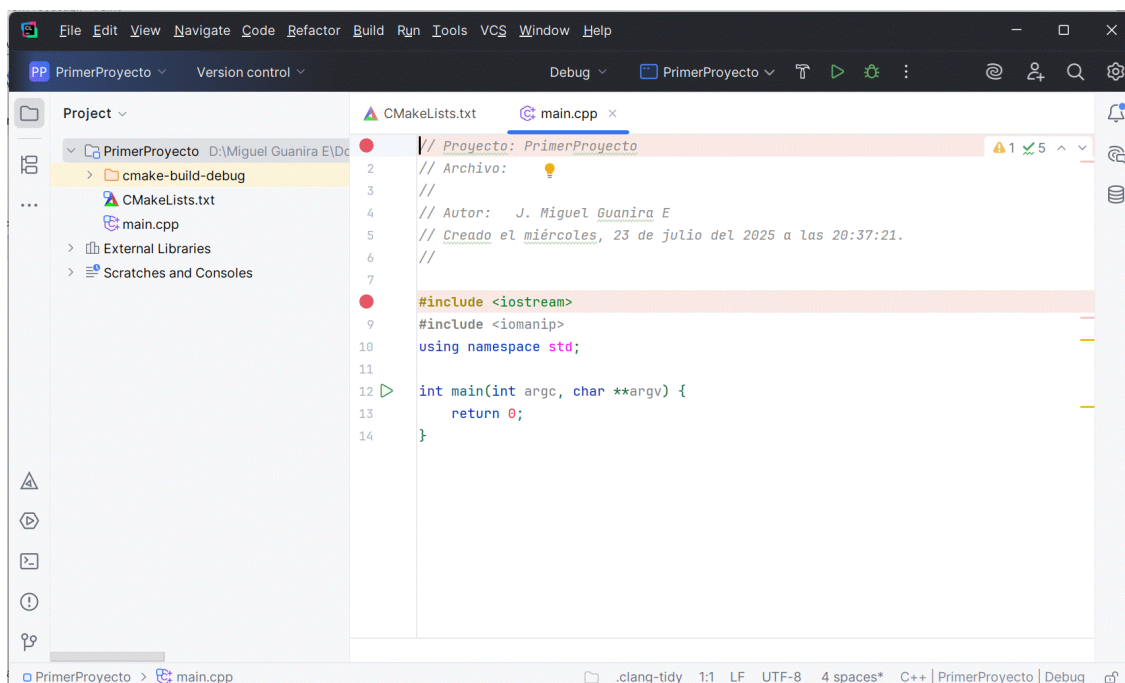
En la carpeta observaremos lo siguiente:



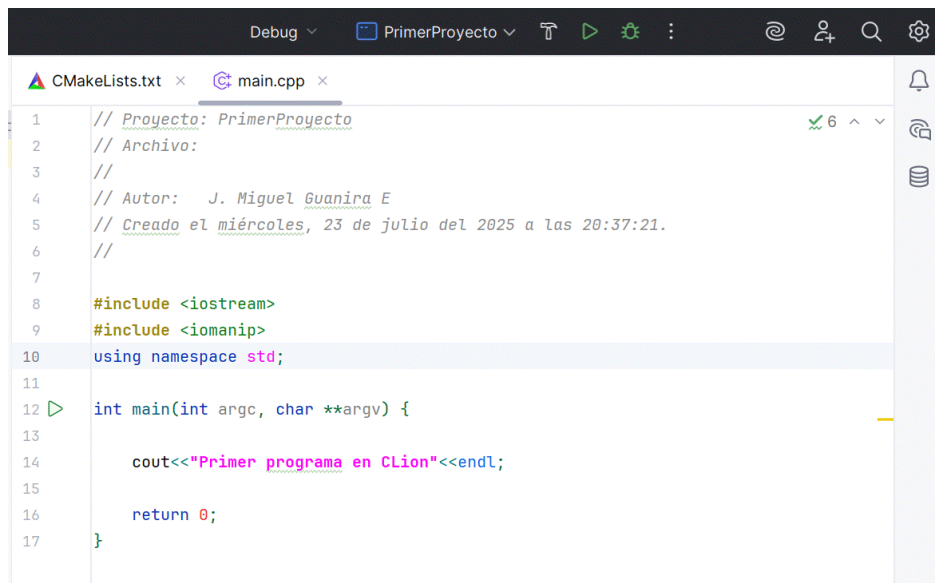
El archivo "main.cpp" es el archivo que contendrá la función "main", los otros archivos y carpetas que aparecen allí, como lo indicamos le servirán al CLion para compilar y crear el archivo ejecutable que vamos a elaborar. Más adelante veremos cómo cambia esta carpeta.

Escribir el programa

Ahora empezaremos a escribir nuestro primer programa en el IDE del CLion, para esto, como se ve a continuación, repitiendo la imagen de la creación del proyecto:



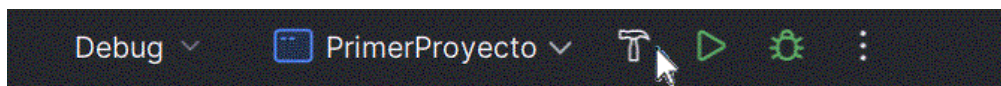
En la parte izquierda de la ventana se aprecia el contenido de la carpeta del proyecto, a la derecha se aprecia el contenido del archivo main.cpp. Lo que vemos es producto de una plantilla que el CLion utiliza para ahorrarnos tiempo en escribir nuestro programa. Como se ve solo tenemos que escribir el código que deseamos sin preocuparnos por agregar órdenes como "include", "return", etc. Entonces escribamos nuestro primer programa, como se ve a continuación:



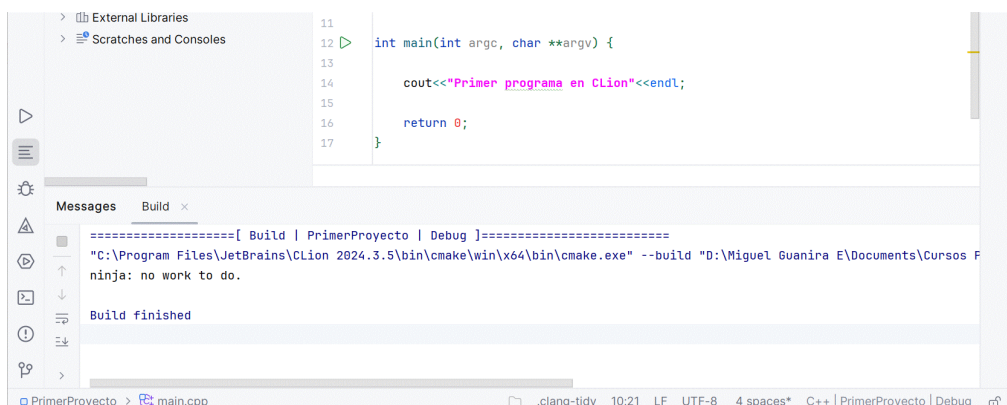
```
1 // Proyecto: PrimerProyecto
2 // Archivo:
3 //
4 // Autor: J. Miguel Guanira E
5 // Creado el miércoles, 23 de julio del 2025 a las 20:37:21.
6 //
7
8 #include <iostream>
9 #include <iomanip>
10 using namespace std;
11
12 int main(int argc, char **argv) {
13
14     cout<<"Primer programa en CLion"<<endl;
15
16     return 0;
17 }
```

Compilación y ejecución de un programa

Para compilar nuestro programa solo debemos presionar el botón denominado "Build" como se ve a continuación:

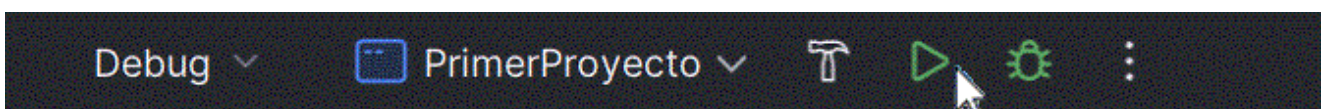


El programa se compilará, y si no hemos cometido algún error veremos en pantalla lo siguiente:



La frase "Build finished" en la ventana inferior indicará que la compilación se ha producido sin problemas.

La ejecución del programa se realizará presionando el botón "Run" como se muestra a continuación:



Al hacerlo veremos la ejecución del programa en una ventana en la parte inferior del IDE, como se muestra a continuación:

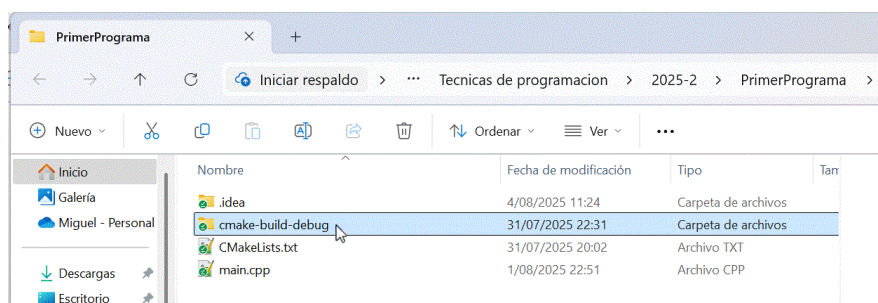


Como se ve en la figura, en la ventana inferior se aprecia el resultado de la ejecución de nuestro programa.

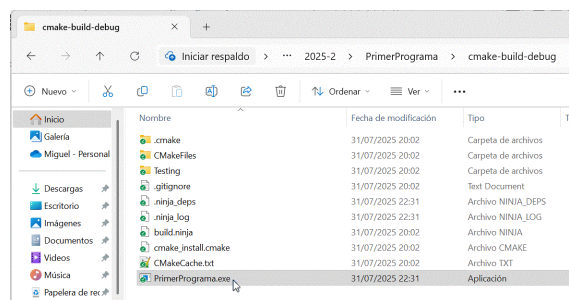
Ejecución de un programa sin necesidad de emplear el IDE

Una segunda forma de ejecutar el programa es, haciéndolo directamente del archivo ejecutable, archivo con extensión .exe o con el ícono , sin la intervención del IDE. Esta forma es importante que la entienda porque usted debe saber que los programas que desarrolle en su vida profesional no serán para usted sino para una persona que lo contrate para que solucione un problema. Esa persona no tiene por qué saber utilizar el CLion, tampoco le interesará leer su código en lenguaje C++, solo le interesa saber cómo ejecutar su programa sin tener siquiera que instalar el CLion. Para realizar esto siga los siguientes pasos¹:

Abra nuevamente el explorador de archivos y ubíquese en la carpeta de su proyecto, si usted ya compiló sin errores el proyecto, verá que el contenido de la carpeta ha cambiado, verá algo similar a lo siguiente:



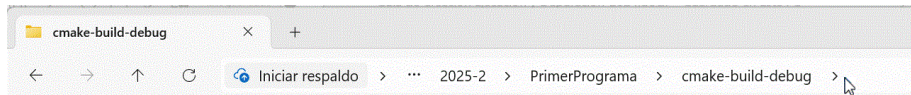
Como ve, se han creado dos carpetas más, la más importante es "cmake-build-debug" que contendrá los archivos que permitieron compilar el proyecto y crear el archivo ejecutable. Abra esa carpeta y podrá observar el archivo ejecutable:



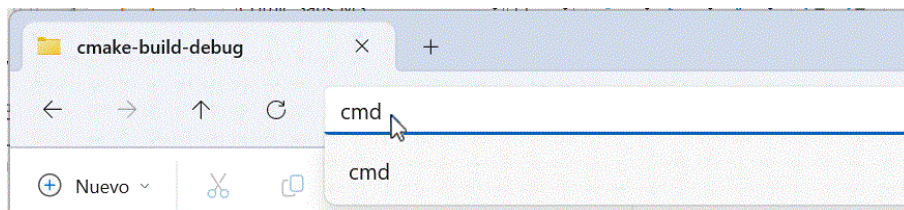
¹ Este proceso solo lo podrá realizar si modificó las variables de entorno como se explica en la "Guía de instalación del CLion"

Como ve, el nombre del programa ejecutable tiene el mismo nombre que el que le dio a su proyecto. Este archivo contiene todo lo necesario para poder ejecutar su programa, esto quiere decir que no depende del CLion para ejecutarlo, por lo tanto, usted puede copiar este archivo a cualquier otra carpeta de su computador y ejecutarlo sin problemas.

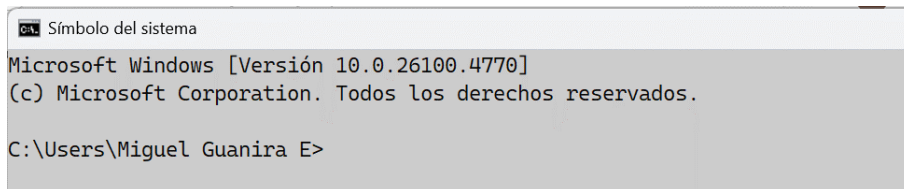
Ahora veremos el proceso para ejecutarlo. Un error que se comete siempre es pensar que dándole dos golpes con el mouse vamos a poder ver el resultado de programa, pues bien, si lo hace, sólo verá una ventana que se abre e inmediatamente se cierra sin darle tiempo a ver el resultado. Para hacerlo correctamente debemos abrir una ventana especial denominada "Ventana de comandos" y allí ejecutar su programa. Para esto diríjase al recuadro donde se muestra la ruta donde se encuentra el archivo ejecutable como se muestra a continuación:



Allí escriba el comando "cmd", como mostramos:



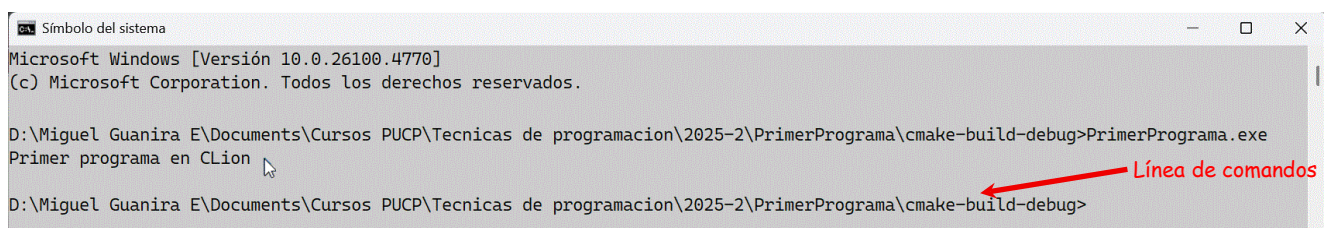
Al presionar la tecla "ENTER" [↵] se abrirá la ventana de comandos como vemos a continuación:



Inmediatamente presione la tecla "TAB" [→], aparecerá en la línea de comandos el nombre del archivo ejecutable, como se ve a continuación:

Como en la carpeta habrá varios archivos, es probable que vea el nombre del primer archivo que se encuentra en la carpeta, si esto sucede vuelva a presionar la tecla "TAB" hasta que aparezca. Si la cantidad de archivos en la carpeta fueran muchos para acelerar el proceso escriba las primeras letras del nombre del archivo y presione "TAB", así aparecerán solo los nombres de los archivos que empiecen con esas letras.

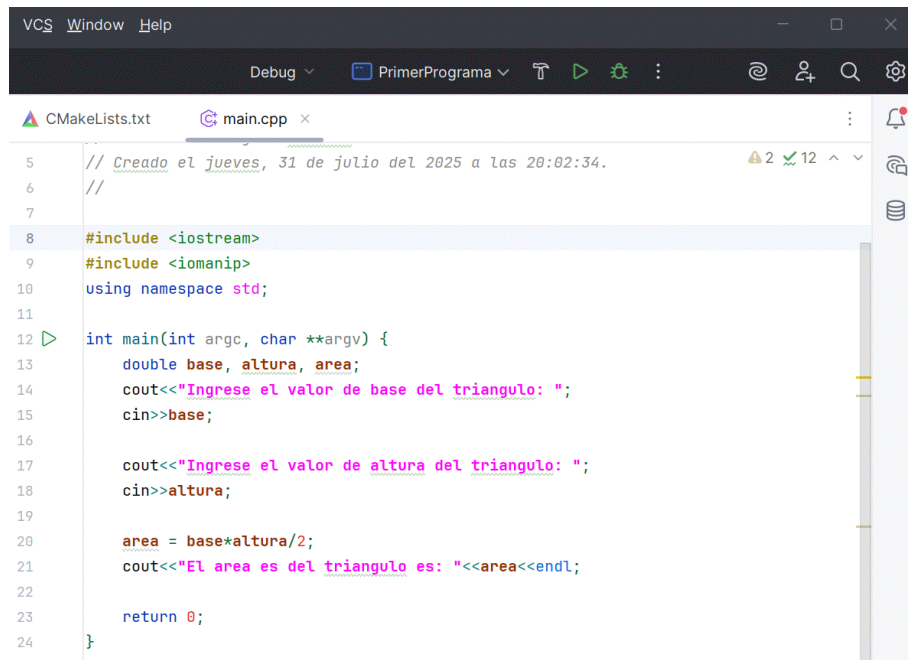
Una vez tengamos el nombre del archivo ejecutable en la línea de comandos presione la tecla ENTER. Inmediatamente usted verá el resultado de la ejecución de su programa como se muestra a continuación:



Como observa el resultado del programa aparece inmediatamente. Una manera de verificar que el programa se ha ejecutado correctamente es que luego de mostrar los resultados el sistema volverá a mostrar a línea de comandos como se ve en la misma figura.

Detección de errores y depuración de un programa en C/C++ usando el CLion

El CLion tiene muchas herramientas que nos permitirán acelerar la escritura de un programa detectando errores que cometamos al escribirlos (errores de sintaxis) o errores lógicos que podamos haber cometido en el algoritmo que hayamos diseñado. Para mostrar estas propiedades crearemos un segundo proyecto y en él escribiremos el programa que se muestra a continuación:

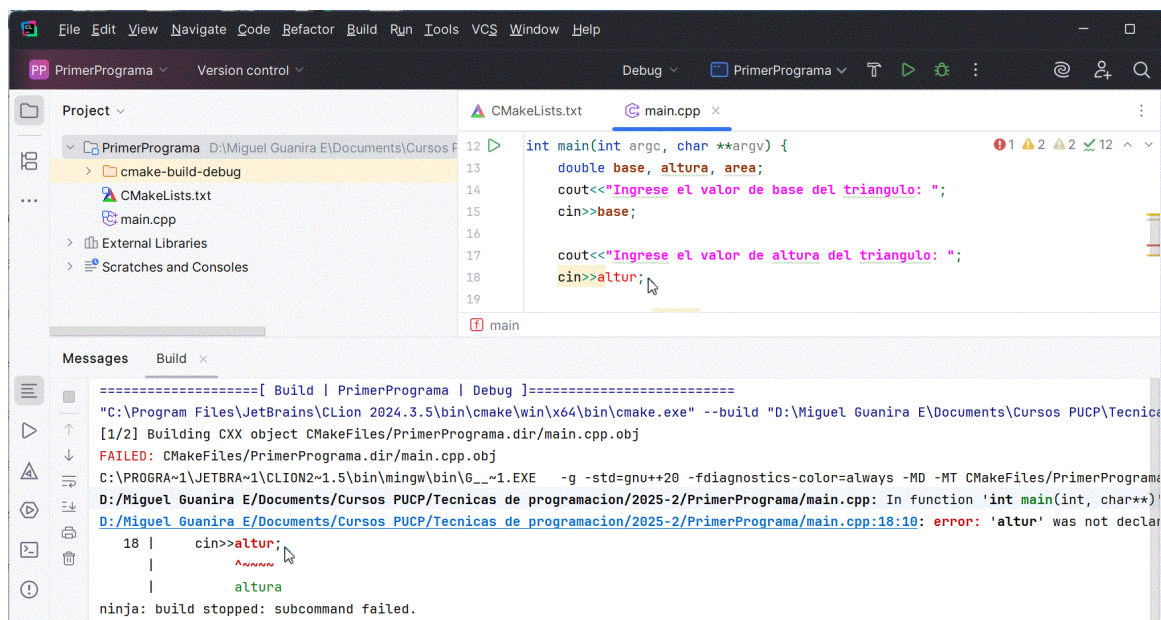


```
5 // Creado el jueves, 31 de julio del 2025 a las 20:02:34.
6 //
7
8 #include <iostream>
9 #include <iomanip>
10 using namespace std;
11
12 int main(int argc, char **argv) {
13     double base, altura, area;
14     cout<<"Ingrese el valor de base del triangulo: ";
15     cin>>base;
16
17     cout<<"Ingrese el valor de altura del triangulo: ";
18     cin>>altura;
19
20     area = base*altura/2;
21     cout<<"El area es del triangulo es: "<<area<<endl;
22
23     return 0;
24 }
```

Lo primero que debe observar es que el CLion, mientras usted escribe el programa, el IDE irá verificando lo que está escribiendo y por inteligencia artificial (IA) le presentará una opción de código, si coincide con lo que quiere hacer solo debe presionar la tecla de tabulación para completar la instrucción, si no coincide solo debe seguir escribiendo la orden.

Por otro lado, si usted comete un error de sintaxis, el IDE subrayará el error o cambiará de color la palabra mal escrita.

Si usted ha cometido algún error, cuando seleccione el botón para compilar le aparecerá en la parte inferior del IDE los errores. Las indicciones de error son bastante claras por lo que le ayudarán a corregir cada error. Un ejemplo se muestra a continuación cuando se borra la letra 'a' de la variable "altura" en la línea 18 del programa, como se muestra a continuación:



```
12 int main(int argc, char **argv) {
13     double base, altura, area;
14     cout<<"Ingrese el valor de base del triangulo: ";
15     cin>>base;
16
17     cout<<"Ingrese el valor de altura del triangulo: ";
18     cin>>altur;
19
20     area = base*altura/2;
21     cout<<"El area es del triangulo es: "<<area<<endl;
22
23     return 0;
24 }
```

Messages Build x

===== [Build | PrimerPrograma | Debug] =====

"C:\Program Files\JetBrains\CLion 2024.3.5\bin\cmake\win\x64\bin\cmake.exe" --build "D:\Miguel Guanira E\Documents\Cursos PUCP\Tecnicas de programacion\2025-2\PrimerPrograma" --target CXX object CMakeFiles/PrimerPrograma.dir/main.cpp.obj

[1/2] Building CXX object CMakeFiles/PrimerPrograma.dir/main.cpp.obj

FAILED: CMakeFiles/PrimerPrograma.dir/main.cpp.obj

C:\PROGRA~1\JETBRA~1\CLION2~1.5\bin\mingw\bin\G_~1.EXE -g -std=gnu++20 -fdiagnostics-color=always -MD -MT CMakeFiles/PrimerPrograma.dir/main.cpp.obj -D/Miguel Guanira E/Documents/Cursos PUCP/Tecnicas de programacion/2025-2/PrimerPrograma/main.cpp -o CMakeFiles/PrimerPrograma.dir/main.cpp.obj

D:\Miguel Guanira E\Documents\Cursos PUCP\Tecnicas de programacion\2025-2\PrimerPrograma\main.cpp:18:10: error: 'altur' was not declared in this scope

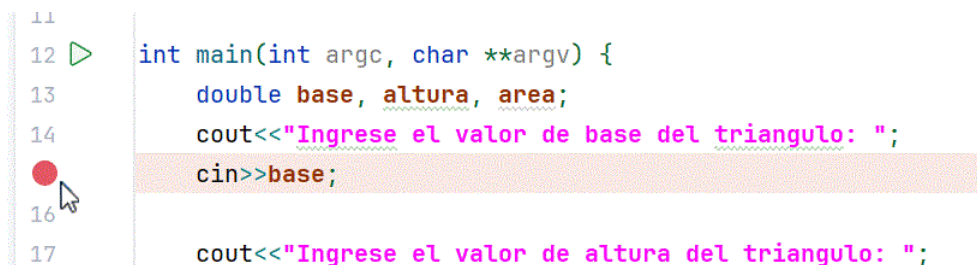
```
18 |     cin>>altur;
   |           ^~~~~
   |           altura
```

ninja: build stopped: subcommand failed.

Cuando pase esto, usted deberá recorrer esta ventana de salida buscando el primer error detectado, presione el botón del mouse sobre la frase en azul e inmediatamente el IDE lo colocará en la línea de su programa con el error. Corrija el error y luego vuelva a la ventana inferior a repetir el proceso con el siguiente error. Una vez haya corregido todos los errores vuelva a compilar el programa, si no se encuentran nuevos errores se generará el programa ejecutable.

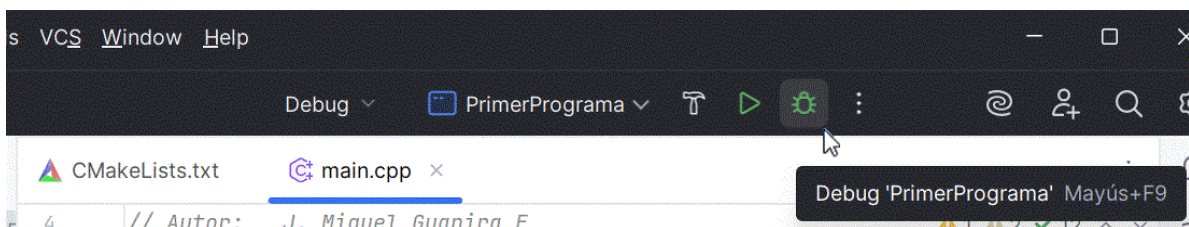
Los errores de lógica son más complicados de localizar, debido a su naturaleza misma el IDE no es capaz de detectarlo. Entienda que un error lógico está referido a un error en el algoritmo que hemos empleado para desarrollar nuestro programa o al haber escrito algo en el programa en lugar de otra cosa, pero que sintácticamente es correcta. Por ejemplo si en vez de + ponemos -, si la fórmula dice que debemos calcular el seno de un ángulo, $\sin(x)$, y en lugar de eso colocamos $\cos(x)$ o en lugar de $\sin(x)$ escribimos $\sin(y)$, etc. el IDE no nos alertará que hayamos cometido un error. Por eso ante la imposibilidad de detectar el error, los entornos de desarrollo implementan herramientas que nos ayudan a encontrarlos. Estas herramientas se denominan "herramientas de depuración" o "Debugger".

Lo primero que debemos hacer antes de depurar un programa es colocar un punto de interrupción (breakpoint), esto es debemos marcar una línea del programa en donde se detendrá la ejecución del programa para empezar la depuración, para esto diríjase a la zona donde se numeran las líneas del código y presione el botón del mouse, como se muestra a continuación:

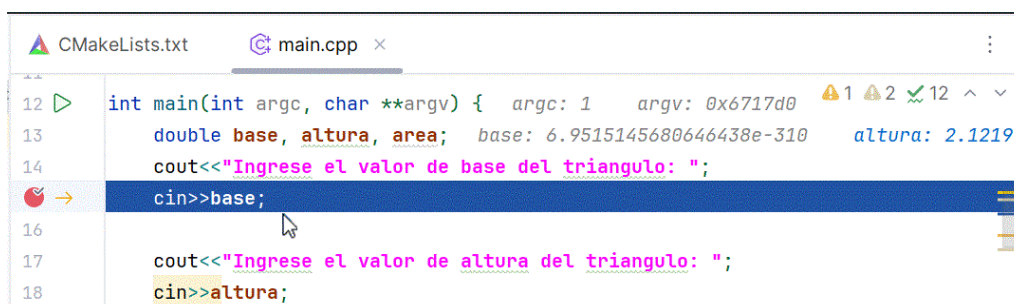


Inmediatamente, como se ve, se reemplazará el número de la línea por un círculo de color rosa, y el fondo de la línea se tornará en color rosa claro, esto indicará que se ha establecido un punto de interrupción.

Una vez hecho esto, debemos empezar el proceso de depuración (Debug), para esto presionemos el botón con la figura de un insecto (Bug) como se muestra a continuación:

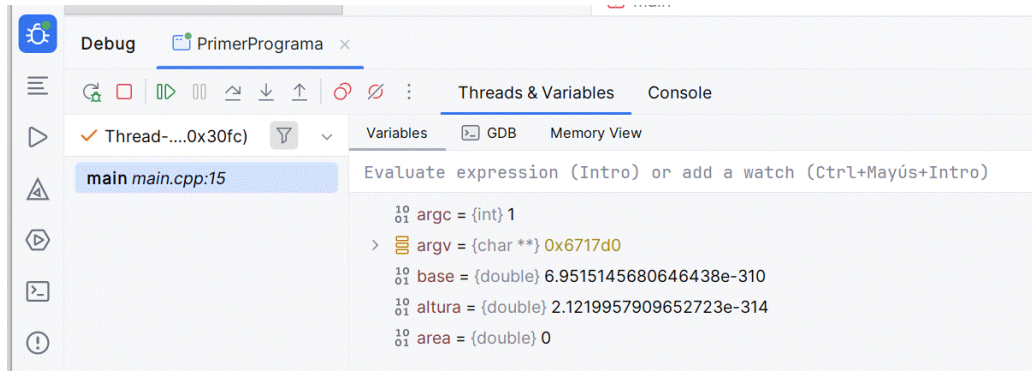


Al hacerlo el programa empezará a ejecutarse y se detendrá en el punto de interrupción (breakpoint) que hayamos colocado, como se muestra a continuación:



La flecha a la izquierda y el fondo azul indican la línea de código que se va a ejecutar. Observe también que el programa verá que aparecen, al lado de las instrucciones, los nombres de las variables con su valor al lado. Recuerde que el C/C++ no inicializa sus variables.

Además de esto usted podrá apreciar que en la parte inferior del IDE aparecerá una ventana similar a la que a continuación se muestra:



En ella se ven, en la parte superior, varios botones, de los cuales los más importantes describiremos a continuación:

Botón de Finalización de la depuración



Como su nombre lo indica, da por terminada la depuración (Stop) del programa en cualquier momento.

Botón Reanudar depuración [F9]



Al presionarlo continuará la ejecución del programa sin detenerse hasta encontrar el siguiente punto de interrupción (Resume Program). Presionando la tecla F9 de su computador podrá realizar la misma labor sin usar la barra de herramientas.

Botón Paso a Paso [F7]



Nos permite ejecutar la línea marcada, deteniéndose la ejecución en la línea siguiente (Step Into). Si la línea contiene una función entonces la ejecución se detendrá en la primera línea de código de la función. La tecla F7 nos permite hacer la misma tarea.

Botón Ejecución Paso a paso por encima [F8]



Si nos encontramos en una línea que contiene una función que nosotros hayamos implementado, este botón permite ejecutar la línea sin depurar la función (Step Over). Esto lo haremos cuando estamos seguros que el código que hemos escrito para esa función es correcto y no queremos perder el tiempo ejecutando línea a línea cada una de sus instrucciones. También se puede usar la tecla F8 de su computador.

En la parte inferior de la ventana se nos muestra cómo va cambiando el valor de las variables conforme vamos avanzando en la depuración de nuestro programa como se aprecia a continuación, recuerde también que el valor de una variable también se aprecia al lado de la instrucción que la contiene:



```
Variables  GDB  Memory View

Evaluate expression (Intro) or add a watch (Ctrl+Mayús+Intro)

10  argc = {int} 1
> 01  argv = {char **} 0x6717d0
10  base = {double} 6.9515145680646438e-310
01  altura = {double} 2.1219957909652723e-314
10  area = {double} 0
```

El proceso de depuración es una tarea muy importante en la elaboración de un programa, es el medio más eficaz para ubicar y corregir los errores lógicos en un programa. Esta tarea se vuelve imprescindible sobre todo empleando un lenguaje de programación como el C/C++ que por su misma naturaleza es muy fácil cometer errores si no se tienen claros los conceptos de programación y no se emplean buenas prácticas de programación.

Usted debe ejercitarse mucho en el uso del depurador hasta dominarlo, de eso dependerá su éxito en la elaboración de programas.

Creación de bibliotecas de funciones y carpetas en un proyecto de CLion

Como verá, los programas que implementará serán cada vez más extensos, por eso es que los entornos de desarrollo integrado ya no hablan de "programas" sino de "proyectos". Estamos hablando de programas que tendrán miles de líneas de código y que por lo tanto un programa no estará compuesto por un solo archivo de código sino por muchos. Por esta razón debemos adquirir buenas prácticas de programación de lo contrario, nuestros programas se volverán ininteligibles, difíciles de depurar e imposibles de actualizar o mejorar. En este documento vamos a empezar estas buenas prácticas explicando la forma que emplear varios archivos de código en un mismo proyecto.

Cuando usted desea emplear en su programa una función como "pow", "abs", "sqrt", etc. lo que hace es incluir el archivo "cmath" en su código (`#include <cmath>`) y así simplemente puede emplear esas funciones sin tener que implementarlas desde cero, el sistema está agregado al programa la biblioteca de funciones "cmath". Pero, qué pasa si las funciones que quiere implementar el C++ no las ha implementado, pues tendremos que implementarlas nosotros mismos. Ahora bien, como primera buena práctica de programación que debe aprender es que en el archivo "main.cpp", como su nombre lo indica, contiene la función "main" de nuestro proyecto, por lo que no debe contener la declaración ni la implementación de funciones como `areaDelTriangulo`, `factorial`, etc., esto se debe hacer en archivos independientes que denominaremos bibliotecas.

Cada biblioteca debe contener dos archivos, uno con la declaración de las funciones (archivo de encabezados o "Header File") cuyo nombre tendrá como extensión ".h" o ".hpp". y otro con la implementación de las funciones (archivo fuentes o "Source File") cuyo nombre tendrá como extensión ".cpp". En cada uno de estos archivos se podrá colocar todas las funciones que se requieran.

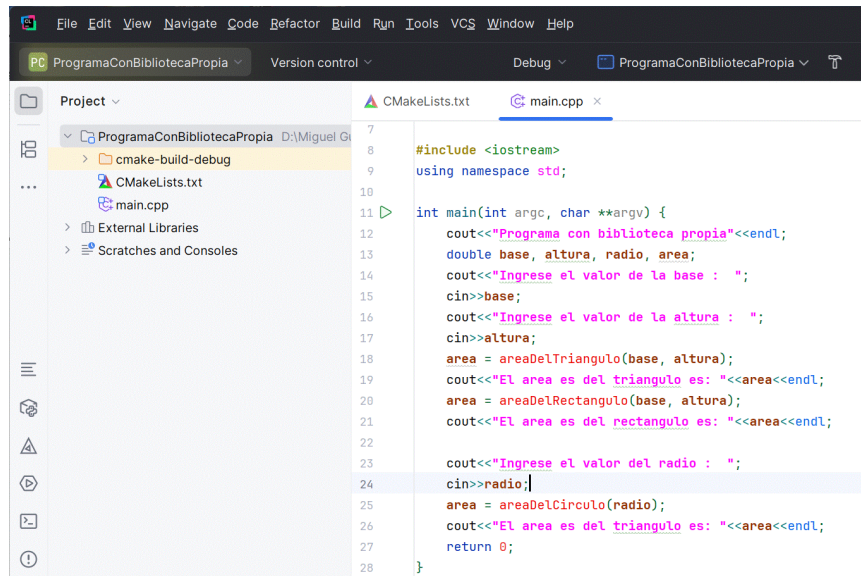
Un proyecto puede tener muchas bibliotecas de funciones con la finalidad de agrupar las funciones de acuerdo con las tareas que realizan. Por ejemplo si tenemos funciones para calcular áreas de superficies podemos crear las funciones: `areaDelTriangulo`, `areaDelCuadrado`, `areaDelCirculo`, etc. dentro de una biblioteca formada por los archivos como `areaDesuperficies.h` y `areaDesuperficies.cpp`, si en el mismo proyecto se requieren funciones como promedio,

desviacionEstandar, promedioPonderado, etc. se deberá crear una biblioteca formada por los archivos como funcionesEstadisticas.h y funcionesEstadisticas.cpp.

El uso de bibliotecas propias de funciones en un programa no solo da una orden al proyecto sino que esas bibliotecas se podrán reutilizar en otros proyectos sin que requiera de implementarlas desde cero de las funciones que contengan..

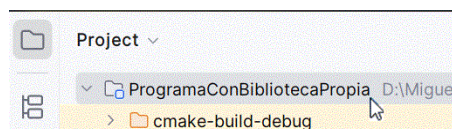
Finalmente, como en un proyecto se podrá emplear muchas bibliotecas de funciones será bueno que éstas no estén desperdigadas en el proyecto por lo que se recomienda que se defina una carpeta dentro del proyecto y allí se coloquen las bibliotecas.

A continuación se mostrará el procedimiento a seguir para crear la carpeta y una biblioteca de funciones. Empiece creando un proyecto como el siguiente:

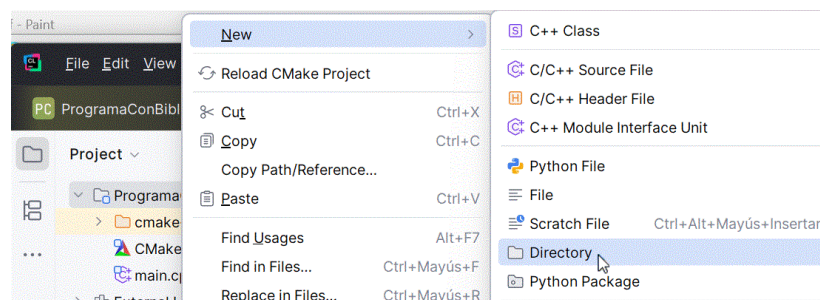


Como ve el programa desea utilizar 3 funciones `areaDelTriangulo`, `areaDelRectangulo` y `areaDelRecangulo`. Si compila o intenta ejecutar el programa, no podrá hacerlo porque el compilador no encuentra la definición ni la implementación de esas funciones, por lo que habrá que implementarlas en una biblioteca.

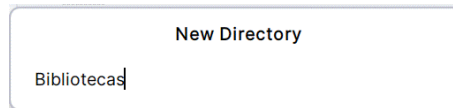
Primero crearemos una carpeta denominada "Bibliotecas", para esto coloque el cursor sobre el nombre del proyecto como se indica a continuación:



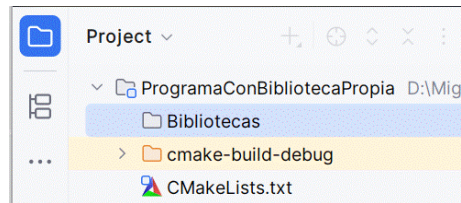
Presione el botón derecho del ratón y en el menú que aparece seleccione la opción "New" y en el siguiente menú la opción "Directory":



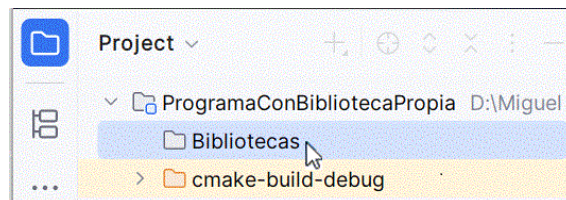
En la ventana que aparezca escriba:



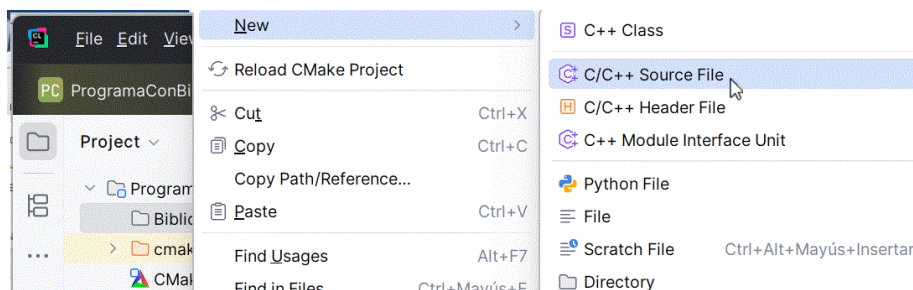
Presione la tecla ENTER y la carpeta se habrá creado en el proyecto:



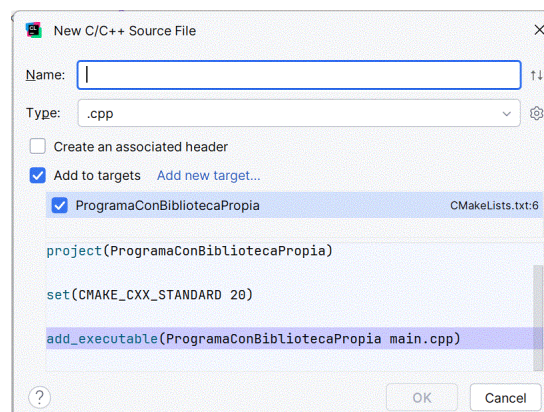
Ahora crearemos la biblioteca, para esto coloque el cursor sobre el nombre de la carpeta creada:



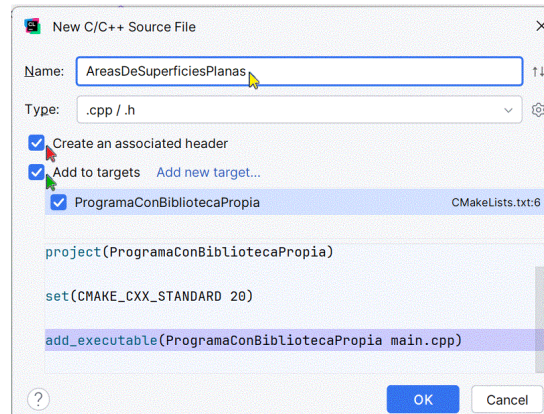
Presione el botón derecho del ratón y en el menú que aparece seleccione la opción "New" y en el siguiente menú la opción "C/C++ Source File":



Allí aparecerá una ventana como la que se aprecia a continuación:



En la casilla "Name" coloque el nombre de la biblioteca, por ejemplo "AreasDeSuperficiesPlanas", luego seleccione la opción "Create an associated header" (para no tener que repetir el proceso para crear el "C/C++ Header File") y finalmente asegúrese que la opción "Add to targets" esté seleccionada (de lo contrario la biblioteca no se enlazará con el proyecto y tendrá que hacer el enlace manualmente). La ventana debe quedar similar a la siguiente imagen:



Al presionar la tecla "OK" se crearan los dos archivos como se aprecia a continuación (observar la flechas amarilla y roja):

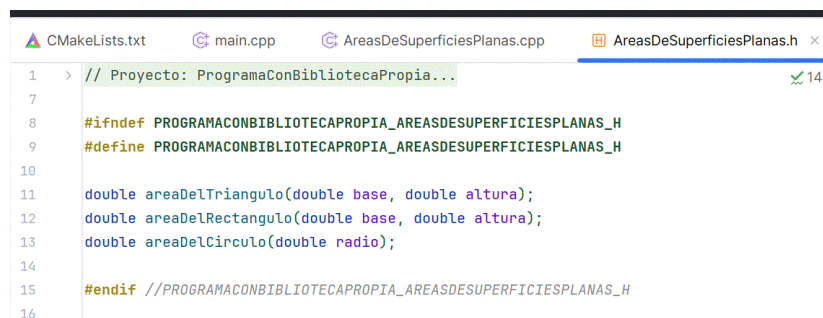


Para verificar si el procedimiento se ha realizado correctamente, verifique el contenido del archivo "CMakeLists.txt" que deberá contener un texto similar al siguiente:



Observe las líneas marcadas, si las dos últimas líneas no aparecen en su proyecto debe ser porque no marcó la opción "Add to targets" al crear los archivos. La forma más sencilla de corregirlo será escribiendo directamente las dos líneas faltantes.

Si ya tiene el archivo completo, entonces podrá seguir con la creación de la biblioteca, para esto vaya al archivo "AreasDeSuperficiesPlanas.h" y allí escriba los encabezados de las funciones que quiere implementar, estas deben hacerse obligatoriamente entre la cláusula "#define" y la cláusula "#end", como se aprecia a continuación:



A continuación complete el archivo "AreasDeSuperficiesPlanas.cpp" con la implementación de las funciones, como se aprecia a continuación:

```

1 > // Proyecto: ProgramaConBibliotecaPropia...
7
8 #include <iostream>
9 #include <iomanip>
10 using namespace std;
11
12 double areaDelTriangulo(double base, double altura) {
13     return base*altura/2;
14 }
15
16 double areaDelRectangulo(double base, double altura) {
17     return base*altura;
18 }
19
20 double areaDelCirculo(double radio) {
21     return 3.141592*radio*radio;
22 }

```

Finalmente, vaya al archivo main.cpp e incluya la biblioteca "AreasDeSuperficiesPlanas.h" en la función main, como se aprecia a continuación:

```

7
8 #include <iostream>
9 using namespace std;
10 #include "Bibliotecas/AreasDeSuperficiesPlanas.h"
11
12 int main(int argc, char **argv) {
13     cout<<"Programa con biblioteca propia"<<endl;
14     double base, altura, radio, area;
15     cout<<"Ingrese el valor de la base : ";
16     cin>>base;
17     cout<<"Ingrese el valor de la altura : ";
18     cin>>altura;
19     area = areaDelTriangulo(base, altura);
20     cout<<"El area es del triangulo es: "<<area<<endl;
21     area = areaDelRectangulo(base, altura);
22     cout<<"El area es del rectangulo es: "<<area<<endl;
23
24     cout<<"Ingrese el valor del radio : ";
25     cin>>radio;
26     area = areaDelCirculo(radio);
27     cout<<"El area es del triangulo es: "<<area<<endl;
28     return 0;
29 }

```

Si todo está bien compile y ejecute el proyecto, con lo que obtendrá el siguiente resultado:

```

Run ProgramaConBibliotecaPropia x
"D:\Miguel Guanira E\Documents\Cursos PUCP\
Programa con biblioteca propia
Ingrese el valor de la base :15.5
Ingrese el valor de la altura :21.3
El area es del triangulo es: 165.075
El area es del rectangulo es: 330.15
Ingrese el valor del radio :5.72
El area es del triangulo es: 102.788
Process finished with exit code 0

```

San Miguel, 1 de agosto del 2025.